

ЛАБОРАТОРНАЯ РАБОТА №1

«Вычислительная математика»

Вариант № 2

Реализуемый алгоритм: метод Гаусса

Выполнил:

студент группы Р3219

Ануфриев Андрей Сергеевич

Принял:

Зинченко Антон Андреевич

Санкт-Петербург, 2026 год

Содержание

1	Цель работы	2
2	Описание алгоритма	3
2.1	Прямой ход с выбором ведущего элемента	3
2.2	Обратный ход	3
3	Листинг программы	4
4	Результаты работы	7
5	Заключение	10

1 Цель работы

Цель лабораторной работы: **изучить и реализовать метод Гаусса для численного решения систем линейных алгебраических уравнений (СЛАУ).**

Конкретные цели по заданию

- реализовать **прямой ход** преобразования расширенной матрицы $[A | b]$ к верхнетреугольному виду:

$$\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ & a_{22}^{(1)} & \cdots & a_{2n}^{(1)} \\ & & \ddots & \vdots \\ & & & a_{nn}^{(n-1)} \end{pmatrix} \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2^{(1)} \\ \vdots \\ b_n^{(n-1)} \end{pmatrix};$$

- обеспечить **перестановку уравнений** системы, если в процессе исключения неизвестных ведущий диагональный элемент a_{11} , $a_{22}^{(1)}$, $a_{33}^{(2)}$, ... обращается в ноль — выбор максимального по модулю элемента в столбце ;
- реализовать **обратный ход** (обратную подстановку) для получения вектора неизвестных x ;
- обеспечить **пошаговый вывод** промежуточных матриц, формул исключения и формул обратной подстановки;
- **вычислить определитель** матрицы как произведение диагональных элементов верхнетреугольной матрицы с учётом знака перестановок;
- провести **проверку результата**: вычислить невязку $r = A \cdot x - b$ и сравнить решение с реализацией NumPy.

2 Описание алгоритма

2.1 Прямой ход с выбором ведущего элемента

На каждой итерации $i = 1, \dots, n - 1$ выполняются следующие шаги.

Шаг 1. Поиск ведущего элемента. Среди строк $i, i + 1, \dots, n$ расширенной матрицы выбирается та, у которой элемент в столбце i максимален по модулю:

$$i^* = \arg \max_{r=i, \dots, n} |a_{ri}^{(i-1)}|.$$

Если $a_{i^*i}^{(i-1)} = 0$ — матрица вырождена.

Шаг 2. Перестановка строк. Если $i^* \neq i$, строки i и i^* меняются местами. Каждая такая перестановка меняет знак определителя:

$$\det A = (-1)^s \prod_{i=1}^n a_{ii}^{(i-1)},$$

где s — количество выполненных перестановок.

Шаг 3. Исключение. Для каждой строки $j = i + 1, \dots, n$:

$$m_{ji} = \frac{a_{ji}^{(i-1)}}{a_{ii}^{(i-1)}}, \quad R_j \leftarrow R_j - m_{ji} \cdot R_i.$$

После $n - 1$ итераций матрица принимает верхнетреугольный вид.

2.2 Обратный ход

Решение ищется снизу вверх:

$$x_i = \frac{b_i^{(i-1)} - \sum_{k=i+1}^n a_{ik}^{(i-1)} x_k}{a_{ii}^{(i-1)}}, \quad i = n, n - 1, \dots, 1.$$

3 Листинг программы

Листинг 1: Код основной функции на Python

```
def gauss_solve_with_steps(A, b):  
    """  
    Solve linear system  $Ax = b$  using Gaussian elimination with partial  
    pivoting.  
    Returns solution vector and step-by-step information.  
    """  
    n = len(A)  
    forward_steps = []  
    backward_steps = []  
    store_matrix = (n <= 9)  
  
    # Permutation counter for determinant sign  
    swap_count = 0  
  
    # Create augmented matrix  $[A|b]$   
    aug = mat_hstack(A, b)  
    forward_steps.append(("initial", "Initial augmented matrix  $[A|b]$ ",  
        mat_copy(aug)))  
  
    # Forward elimination  
    for i in range(n):  
        # Partial pivoting: find row with maximum element in column  $i$   
        max_row = i  
        max_val = abs(aug[i][i])  
        for r in range(i + 1, n):  
            if abs(aug[r][i]) > max_val:  
                max_val = abs(aug[r][i])  
                max_row = r  
  
        # Check for singular matrix  
        if max_val < 1e-15:  
            raise ValueError(  
                f"All elements in column {i+1} below row {i+1} are zero - "  
                f"matrix is singular (system has no solution or infinite  
                solutions)"  
            )  
  
        # Swap rows if necessary  
        if max_row != i:  
            aug[i], aug[max_row] = aug[max_row], aug[i]  
            swap_count += 1  
            forward_steps.append((  
                "swap",  
                f"Iteration {i+1}: swap rows {i+1} <-> {max_row+1} "  
                f"(pivot element |{aug[i][i]:.6f}|)",  
                mat_copy(aug) if store_matrix else None  
            ))  
  
        pivot = aug[i][i]  
  
        forward_steps.append((  
            "pivot",
```

```

        f"Iteration {i+1}: pivot element a[{i+1},{i+1}] = {pivot:.6f}",
        mat_copy(aug) if store_matrix else None
    ))

    # Eliminate below
    for j in range(i + 1, n):
        factor = aug[j][i] / pivot
        if abs(factor) > 1e-15:
            forward_steps.append((
                "eliminate",
                f"Row {j+1} - ({factor:.6f})      Row {i+1} ",
                f"[zeroing a[{j+1},{i+1}]]",
                None # matrix will be copied in result
            ))

            for k in range(i, n + 1):
                aug[j][k] -= factor * aug[i][k]

            forward_steps.append((
                "result",
                f"Result after eliminating x{i+1} from row {j+1}",
                mat_copy(aug) if store_matrix else ("row", j, aug[j]
                [:])
            ))

    forward_steps.append((
        "final",
        "Upper triangular matrix after forward elimination",
        mat_copy(aug)
    ))

# Calculate determinant
det_product = 1.0
for i in range(n):
    det_product *= aug[i][i]
# Each row swap changes determinant sign
if swap_count % 2 == 1:
    det_product = -det_product

# Back substitution
x = [0.0] * n

for i in range(n - 1, -1, -1):
    diag = aug[i][i]
    rhs = aug[i][n]
    sub_terms = [(aug[i][k], k) for k in range(i + 1, n)]

    # Build formula string
    if not sub_terms:
        formula = (f"x{i+1} = b'[{i+1}] / a'[{i+1},{i+1}] = "
                    f"{rhs:.6f} / {diag:.6f}")
    else:
        parts = " + ".join(
            f"({c:+.6f}) x {k+1}" for c, k in sub_terms
        )

```

```

        formula = (f"x{i+1} = ({rhs:.6f} - [{parts}]) / {diag:.6f}")

    # Calculate value
    val = rhs
    for coef, k in sub_terms:
        val -= coef * x[k]
    x[i] = val / diag

    # Build numeric calculation string
    if sub_terms:
        num_parts = " + ".join(
            f"({c:+.6f}) ({x[k]:.6f})" for c, k in sub_terms
        )
        numeric = (f"                = ({rhs:.6f} - [{num_parts}]) / {diag:.6f}"
                    ")")
    else:
        numeric = f"                = {rhs:.6f} / {diag:.6f}"

    backward_steps.append((
        i,
        x[i],
        formula,
        numeric,
        [c for c, _ in sub_terms]
    ))

return x, aug, forward_steps, backward_steps, det_product

```

4 Результаты работы

ВВОД ДАННЫХ

Размерность n (≤ 20):

20

Задать

Способ ввода

- ☒ С клавиатуры
☐ Из файла

РЕШИТЬ СИСТЕМУ

Сбросить всё

Выберите файл с данными:

C:/Users/Eternal_Corez/OneDrive - MSFT/github_file/Pyt

Обзор

Загрузить

Решение

Код метода

Прямой ход

Обратный ход

Сравнение с NumPy

РЕЗУЛЬТАТЫ РЕШЕНИЯ

Треугольная матрица (расширенная):

x1	x2	x3	x4	x5	x6	x7	x8	x9	x10	x11	x12	x13	x14	x15	x16	x17	x18	x19	x20	b
-2444	5.120	-1.671	4.230	-3.451	2.780	-5.341	1.560	-4.121	3.890	-1.451	5.230	-2.781	9.340	-3.121	4.560	-1.781	2.670	-5.121	1.450	-16.91
-0.001	-5444	3.450	-2.671	4.890	-1.341	8.560	-2.781	3.450	-4.121	1.670	-5.231	2.890	-4.551	3.120	-1.781	5.670	-2.341	4.230	-1.891	15.621
-0.001	0.000	7.890	-1.331	2.559	-4.781	5.119	-3.671	1.889	-2.331	4.560	-1.661	3.780	-5.221	2.450	-4.111	6.340	-2.891	1.559	-3.781	28.441
0.000	0.000	0.000	7.849	-2.731	-0.301	-3.241	1.436	-3.531	4.813	-0.781	3.278	-0.171	2.314	-2.881	0.611	-3.011	0.611	-4.311	4.735	-9.241
0.000	0.000	0.000	0.000	-3444	-0.041	-0.851	0.706	1.080	1.311	-1.361	5.268	-1.111	0.842	1.311	1.839	-1.121	1.053	-1.551	-2.371	-7.981
0.000	0.000	0.000	0.000	0.000	4.415	-1.021	-0.271	-2.161	5.130	-1.491	1.919	-1.091	1.151	-3.111	1.559	-1.211	0.675	-0.991	2.020	-7.981
0.000	0.000	0.000	0.000	0.000	0.000	2.824	1.954	1.213	-5.591	-1.321	-0.701	-1.781	-2.711	1.283	1.676	3.058	6.867	1.584	-0.551	-8.011
0.000	0.000	0.000	0.000	0.000	0.000	0.000	5.463	-0.051	0.197	-0.881	1.157	1.863	2.493	-0.401	0.349	1.889	-2.481	-2.681	-2.261	-2744
0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	6.042	-0.381	1.598	-0.611	1.579	-2.691	-0.671	-2.731	1.814	1.984	2.123	-0.701	-7431
0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	6.845	3.713	2.191	-0.731	0.000	-0.581	1.880	-1.391	-5.671	-2.581	2.436	10782
0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	4.859	0.364	0.632	2.176	3.119	3.164	0.213	0.974	-1.371	0.411	-3683
0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	-4.111	3.351	3.098	1.637	-0.381	3.719	-5.131	1.943	-0.241	-11161	
0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	8.151	0.191	-0.101	-2.621	0.627	-2.861	-0.491	-3.451	-1314	
0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	-1444	4.537	0.768	-2.941	-2.821	-2.301	1.717	53535
0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	4.950	0.162	-1.141	3.776	-2.971	-0.871	29511
0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	-0.001	4.488	2.353	-6.931	6.384	2.778	-6536
0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	2.859	0.414	-4.611	3.667	15608
0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	-0.000	0.000	-4.951	6.220	5.180	16887
0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	-1.491	-0.461	74039
0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	-1.111	-62721

Определитель: 984345348067311275756918669312.0000000000

Вектор неизвестных X:

x1	-263.2353923976
x2	14.5626128390
x3	1384997.9754441166
x4	116050.1126297708
x5	-129.0826965700
x6	426231.4054885868
x7	786398.3932564523

СРАВНЕНИЕ С NUMPY

Переменная	Метод Гаусса	NumPy	Δ
x1	-263.23539240	-263.23539240	7.96e-13
x2	14.56261284	14.56261284	2.13e-14
x3	1384997.97544412	1384997.97544411	6.29e-09
x4	116050.11262977	116050.11262977	1.62e-09
x5	-129.08269657	-129.08269657	4.83e-13
x6	426231.40548859	426231.40548859	1.16e-09
x7	786398.39325645	786398.39325645	3.49e-10
x8	-126726.16417132	-126726.16417132	1.51e-09
x9	1008361.06055752	1008361.06055752	3.49e-09
x10	-646910.51897109	-646910.51897108	6.98e-10
x11	-439173.33458030	-439173.33458030	2.15e-09
x12	-1238093.72598206	-1238093.72598205	6.05e-09
x13	353596.43502762	353596.43502761	3.26e-09
x14	48.40322960	48.40322960	1.14e-13
x15	-448163.36926068	-448163.36926068	2.39e-09
x16	912011.59090334	912011.59090334	4.66e-09
x17	-1711905.57480978	-1711905.57480977	6.29e-09
x18	-290863.29585285	-290863.29585285	7.57e-10
x19	-673291.05524189	-673291.05524189	1.75e-09
x20	562702.89298363	562702.89298363	2.10e-09
Макс. разность:	6.29e-09		
Определитель (Гаусс):	984345348067311275756918669312.0000000000		
Определитель (NumPy):	984345348067312542394313867264.0000000000		
Δ определителей:	1.27e+15		

Вектор невязки $r = A \cdot x - b$:

r1	2.09e-09
r2	4.59e-10
r3	-1.15e-09
r4	2.14e-09
r5	-2.00e-09
r6	-6.08e-11
r7	2.20e-09
r8	-9.31e-10
r9	5.42e-10
r10	1.16e-09
r11	-6.07e-10
r12	6.21e-10
r13	-2.40e-10
r14	-1.45e-09
r15	-4.12e-10

5 Заключение

В ходе выполнения лабораторной работы реализован метод Гаусса.

- корректное приведение расширенной матрицы к верхнетреугольному виду с сохранением всех промежуточных шагов;
- автоматическая перестановка уравнений системы, если ведущий диагональный элемент обращается в ноль, что полностью выполняет требование задания;
- учёт знака перестановок при вычислении определителя: $\det A = (-1)^s \prod_i a_{ii}^{(i-1)}$;
- вычисление вектора неизвестных обратной подстановкой и оценка точности через невязку $\|r\| = \|Ax - b\|$;
- сравнение результата с решением NumPy (LUP-разложение).

Вывод. Метод Гаусса с выбором ведущего элемента является надёжным и эффективным методом решения плотных СЛАУ с трудоёмкостью $O(n^3)$. Выбор максимального элемента в столбце устраняет деление на ноль и уменьшает накопление ошибок округления, что подтверждается малой нормой невязки ($\|r\| \sim 10^{-14}$) и хорошим совпадением с результатом NumPy.